# SCS2314 Middleware Architecture: Group Assignment

## Building a Fault-Tolerant Distributed System

## Overview

In groups of four, you will design, build and evaluate a distributed system that keeps working when nodes fail, clocks drift and the network splits. The domain and architecture are your choice; the four core mechanisms below are not.

Each member owns one mechanism: time and ordering, consensus, fault tolerance, or replication and consistency. The four parts must work together in one system. You are assessed on the group system and on your own part.

This assignment assesses LO5 and draws on LO1–LO3.

## Member roles

Each member owns one role and must be able to explain and defend it alone in the viva. Members should still review and test each other's parts.

| Role | What you build | What you must show |
|---|---|---|
| 1. Time and ordering | Clock synchronisation (Cristian, Berkeley or NTP-style) and/or logical clocks (Lamport or vector) used to order events across nodes | Events are ordered correctly under clock skew; what goes wrong without your mechanism; the overhead it adds |
| 2. Consensus | Leader election and agreement among nodes, e.g. Raft-style election and log replication, or a justified simpler protocol | Exactly one leader per term; a new leader after the old one fails; no committed decision is lost or contradicted |
| 3. Fault tolerance | Failure detection (heartbeats, timeouts), failover, redundancy, and recovery of a restarted node | The system keeps serving after a node crash; how long detection and failover take; how a node rejoins and catches up |
| 4. Replication and consistency | A replication strategy (primary-backup, quorums or similar) and a chosen consistency model | Why that consistency model suits your domain; what clients observe during failures and partitions; how conflicts are handled |

The roles overlap by design. For example, consensus depends on failure detection, and replication may rely on the leader. Agree the interfaces between roles early, and describe them in your report.

## Architecture and requirements

Choose the architecture that best fits your domain, and justify the choice in your report. No option scores higher by default; the justification and execution are what count.

| Architecture | Typical shape | Where the roles get interesting |
|---|---|---|
| Client-server | Clients talk to a replicated cluster of 3–5 servers | Leader-based consensus and replication; failover hidden from clients |
| Peer-to-peer | All nodes are equal; data is spread across peers (e.g. consistent hashing, gossip) | No fixed leader; consensus among peers; eventual consistency and conflict handling |
| Hybrid | Some nodes coordinate (super-peers, a metadata cluster), others store or compute | Separate failure handling for coordinators and workers; mixed consistency levels |

**Minimum requirements**

- At least 3 nodes running as separate processes or containers; 5 is recommended where you use majority-based consensus.
- Nodes communicate over the network using gRPC, REST or message-oriented middleware.
- A client application (command-line tool, simple UI or script) that sends requests to the system. It must be able to run as several concurrent clients and log what each one observes (successful requests, errors, values read) with timestamps. A polished interface is not needed.
- A way to inject faults on demand: stop a node, delay or drop messages, partition the network, skew a node's clock.
- Logs or a simple dashboard that make each node's state visible during the demo.
- A reproducible setup, with setup instructions in the repository that get the system running from scratch. Docker Compose is recommended: one command starts every node, and stopping or disconnecting a container is an easy way to inject faults.

## Sample scenarios

The brief is open-ended: pick one of these, adapt it, or propose your own. Your own domain is welcome if all four roles have real work to do in it.

| Scenario | Natural architecture | The hard part |
|---|---|---|
| Replicated key-value store | Client-server | Strong vs eventual consistency under partitions; leader failover without losing writes |
| Payment service | Client-server | Each payment processed exactly once, even if a node fails after debiting but before confirming |
| Event ticket booking | Client-server or hybrid | Never selling the same seat twice when nodes fail mid-booking |
| Distributed file storage | Hybrid (metadata cluster + storage peers) | Keeping metadata consistent while storage nodes come and go |
| Group chat or messaging | Client-server or P2P | Message ordering across users and devices; delivery when a server fails |
| Collaborative document editor | Client-server or P2P | Concurrent edits, causal ordering and conflict resolution |
| Distributed task scheduler | Client-server | Each job runs exactly once, even if a worker or the scheduler crashes |
| IoT sensor aggregation | Hybrid (edge gateways + central cluster) | Timestamps from devices with drifting clocks; gateways that go offline |
| Mini banking ledger | Client-server | Transfers that stay correct across failures; the balance never goes negative |

Whatever you choose, each role must face a real problem in your domain. A scenario where ordering or consistency never matters will limit the marks available to that member.

## Deliverables and demonstration

| Deliverable | Contents |
|---|---|
| Git repository | Source, setup and fault-injection scripts, setup instructions; commit history showing each member's work |
| Final report as a PDF (group part plus one section per member) | Architecture and why it was chosen; how the four roles fit together; results of the fault tests; each member's section covers the design, trade-offs and evaluation of their role |
| Video on YouTube (unlisted is fine) | Up to 15 minutes: the system running and the fault-injection tests below, with each member presenting their own role |
| Demonstration and viva | After submission: live demo under faults, then individual questions |

**Submission in week 12:** upload one zip file containing two files: a README and the report PDF. The README contains each member's name, index number, email and role, the Git repository link and the YouTube video link. The code stays in the repository.

**Incomplete submissions:** any missing component results in an incomplete submission and zero marks.

**Fault-injection demo**

The examiner will ask you to run some or all of these during the demo, and each member explains what happened in their part:

1. Kill the current leader or primary while clients are writing.
2. Partition the network into a majority and a minority, then heal it.
3. Skew one node's clock by several seconds.
4. Restart a failed node and show it catching up.
5. Send concurrent conflicting requests from two clients.

The report and video should present measured results for these tests, such as failover time, data lost or duplicated, and ordering errors, not just a description.

## Marking rubric

The assignment is marked out of 100: 40 marks for the group and 60 for each member individually. Every member receives the same group mark.

| Part | Criterion | Marks |
|---|---|---|
| Group | G1. Architecture and design: fit to the domain, justified choices, clear interfaces between roles | 10 |
| Group | G2. Integration: the four mechanisms work together in one running system | 15 |
| Group | G3. Fault testing: the system survives the demo faults; measured results reported | 10 |
| Group | G4. Report and presentation: clear, accurate, well structured | 5 |
| Individual | I1. Design and correctness of your role | 25 |
| Individual | I2. Evaluation of your role under failures, with evidence and trade-offs | 15 |
| Individual | I3. Viva: understanding of your role and how it connects to the others | 20 |

**Performance levels**

| Criterion | Excellent (80–100%) | Good (65–79%) | Satisfactory (50–64%) | Poor (below 50%) |
|---|---|---|---|---|
| G1 Architecture | Choice clearly justified against alternatives; interfaces precise | Sound choice, some justification | Workable choice, little justification | Unclear or unsuitable design |
| G2 Integration | All four mechanisms interact correctly in one system | Mostly integrated; minor gaps | Parts work separately, weak integration | Parts missing or not working |
| G3 Fault testing | All demo faults handled; results measured and explained | Most faults handled; some measurement | Some faults handled; little evidence | System fails under basic faults |
| G4 Report | Concise, accurate, well argued | Clear with minor gaps | Descriptive, limited analysis | Incomplete or inaccurate |
| I1 Role design | Correct, well reasoned design that handles edge cases | Correct for normal cases, some edge cases | Basic version works, edge cases ignored | Incorrect or not implemented |
| I2 Role evaluation | Trade-offs analysed with measured evidence | Some evidence and discussion | Mostly descriptive | No meaningful evaluation |
| I3 Viva | Explains design, failures and alternatives confidently; links to other roles | Explains own role well, some links | Explains basics with prompting | Cannot explain own work |

## Rules

- **Implement the core yourselves.** Libraries are fine for networking, serialisation and containers.
- **Frameworks need your own contribution.** You may build on an existing framework, for example ZooKeeper, etcd or Consul for coordination, or libp2p, a Kademlia DHT library, or Serf or memberlist for gossip in peer-to-peer designs. If you do, every member must still make their own contribution to their role on top of it, such as a recipe or extension built on the framework and a thorough evaluation under faults. State clearly what each member built and what the framework provides. Kubernetes and similar orchestration frameworks are not allowed.
- **Any language is allowed**, provided the system runs from your setup instructions on a standard machine.
- **Commit history** should show each member's work. Unbalanced histories will be discussed in the viva.